

Domain Driven Design

Building Microservice Systems

Sharif University of Technology, Spring 2026

Ali Abrishami

Seyyed Hossein Mirhosseini



Implementing Simple Business Logic



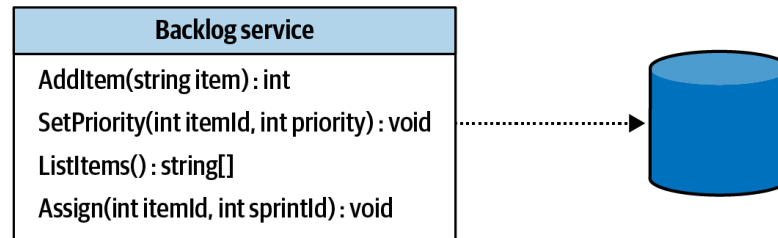
Implementing Simple Business Logic

- Business logic is the **most important** part of software.
- It's **the reason** the software is being implemented in the first place.
- We begin our exploration of the different ways to model and implement business logic code.



Transaction Script

- **Transaction script:** *Organizes business logic by procedures where each procedure handles a single request from the presentation.* (martin fowler)
- A system's public interface can be seen as a **collection** of business transactions that consumers can execute (below figure).
- These transactions can retrieve information managed by the system, modify it, or both.



Transaction script interface



Transaction Script (cont.)

- The pattern organizes the system's business logic based on procedures.
- Each procedure implements an operation that is executed by the system's consumer via its public interface.
- Despite the simplicity, the transaction script pattern is **ubiquitous** in software development.
- The only requirement procedures have to fulfill is **transactional behavior**.
 - *Each operation should either **succeed** or **fail** but can never result in an invalid state.*
- The transaction script pattern is a *foundation* for the more advanced business logic implementation patterns.



Transaction Script Example

C# Example

```
DB.StartTransaction();

var job = DB.LoadNextJob();
var json = LoadFile(job.Source);
var xml = ConvertJsonToXml(json);
WriteFile(job.Destination, xml.ToString());
DB.MarkJobAsCompleted(job);

DB.Commit();
```

Python Example

```
@app.post("/process-next-job")
def process_next_job(db: Session = Depends(get_db)):
    db.begin()
    try:
        job = load_next_job(db)
        json_data = load_file(job.source)
        xml_data = convert_json_to_xml(json_data)
        write_file(job.destination, xml_data)
        mark_job_as_completed(db, job)
        db.commit()
        return {"status": "done", "job_id": job.id}
    except Exception:
        db.rollback()
        raise HTTPException(status_code=500, detail="Job
processing failed")
```



Lack of Transactional Behavior

Without transaction behavior

```
public class LogVisit
{
    ...
    public void Execute(Guid userId, DateTime visitedOn)
    {
        _db.Execute("UPDATE Users SET last_visit=@p1 WHERE
user_id=@p2",
            visitedOn, userId);
        _db.Execute(@"INSERT INTO VisitsLog(user_id,
visit_date)
            VALUES(@p1, @p2)", userId, visitedOn);
    }
}
```

Adding transactional behavior

```
public void Execute(Guid userId, DateTime visitedOn)
{
    try
    {
        _db.StartTransaction();
        _db.Execute(@"UPDATE Users SET last_visit=@p1
            WHERE user_id=@p2",
            visitedOn, userId);
        _db.Execute(@"INSERT INTO VisitsLog(user_id,
            visit_date)
            VALUES(@p1, @p2)",
            userId, visitedOn);

        _db.Commit();
    } catch {
        _db.Rollback();
    }
}
```



Idempotency in Transaction Scripts

- An operation is **idempotent** if performing it **multiple times** has **the same effect** as performing it once.
- Since **transaction scripts** often perform direct business actions (e.g., creating an order, updating a balance), they frequently run inside **distributed or fragile environments** — HTTP calls, retries, network errors, etc.
- If your script is retried because of a timeout or partial failure:
 - **charged twice**
 - **order created twice**
- So, idempotency ensures **safety under retries**.



Idempotency in Transaction Scripts (cont.)

Without idempotency

```
public class LogVisit
{
    ...
    public void Execute(Guid userid)
    {
        _db.Execute("UPDATE Users SET visits=visits+1
WHERE user_id=@p1",
        userid);
    }
}
```

Idempotent implementation

```
public class LogVisit
{
    ...
    public void Execute(Guid userId, long visits)
    {
        _db.Execute("UPDATE Users SET visits = @p1 WHERE
user_id=@p2",
        visits, userId);
    }
}
```



When to Use Transaction Script

- The transaction script pattern is well adapted to the most **straightforward problem** domains in which the business logic resembles **simple** procedural operations.
- The transaction script pattern naturally fits **supporting subdomains** where, by definition, the business logic is **simple**.
 - For example, in extract-transform-load (ETL) operations.
- It can also be used as an adapter for integration with external systems
 - For example, **generic subdomains**, or as a part of an **anticorruption layer**.
- Transaction script should **never** be used for core subdomains.



Active Record

- **Active Record:** *An object that **wraps** a row in a database table or view, encapsulates the database access, and adds domain logic on that data.*
(Martin Fowler)
- Like the transaction script pattern, active record supports cases where the business logic is **simple**.
- Here, however, the business logic may operate on **more complex** data structures.



Implementation

- This pattern uses dedicated objects, known as **active records**, to represent **complicated** data structures.
- These objects also implement data access methods for creating, reading, updating, and deleting records—the so-called **CRUD** operations.
- The difference between the two patterns is that in this case, instead of accessing the database directly, the transaction script manipulates active record objects.



Active Record Example

- The pattern's goal is to encapsulate the complexity of mapping the in-memory object to the database's schema.
- The active record objects can contain **business logic**
 - For example, validating new values assigned to the fields.

```
public class CreateUser
{
    ...
    public void Execute(userDetails)
    {
        try
        {
            _db.StartTransaction();
            var user = new User();
            user.Name = userDetails.Name;
            user.Email = userDetails.Email;
            user.Save();
            _db.Commit();
        } catch {
            _db.Rollback();
            throw;
        }
    }
}
```



When to Use Active Record

- Accordingly, as in the case of the transaction script pattern, the active record pattern lends itself to:
 - Supporting subdomains,
 - Integration of external solutions for generic subdomains,
 - Or model transformation tasks.
- Django's ORM is **inspired** by the Active Record pattern, but it is **not a pure** Active Record implementation.
It mixes ideas from **Active Record** and **Data Mapper**.



Tackling Complex Business Logic



Domain Model

- The domain model pattern was introduced initially in Martin Fowler's book *Patterns of Enterprise Application Architecture*.
- Evans presented a set of patterns aimed at tightly relating the **code** to the **underlying model** of the business domain: *aggregate*, *value objects*, *repositories*, and others.
- The pattern is “**domain model**,” and the *aggregates* and *value objects* are its **building blocks**.



Domain Model (cont.)

- A **Domain** represents the **problem space** or **business area** that the software is trying to solve.
- The domain model pattern is intended to cope with cases of **complex business logic**.
- A Domain Model is a **conceptual** representation of the business domain.
- Here, instead of CRUD interfaces, we deal with complicated **state transitions**, **business rules**, and invariants.
 - Rules that have to be protected at all times.



Domain Model Implementation

- A domain model is an **object model** of the domain that incorporates both **behavior** and **data**.
- DDD's tactical patterns are the building blocks of such an object model:
 1. Value objects
 2. Aggregates
 3. Domain events
 4. Domain services
- All of these patterns share a common theme: they put the **business logic first**.



Domain Model Implementation (cont.)

- The model should be **devoid** of any infrastructural or technological concerns.
 - Such as implementing calls to databases or other external APIs.
- This restriction requires the model's objects to be ***plain old objects***.
 - Objects implementing business logic without relying on or directly incorporating any infrastructural components or frameworks.
- This pattern allows the code to “speak” the **ubiquitous language** and to follow the domain experts' mental models.
- Let's look at the central domain model building blocks, or tactical patterns: **value objects, aggregates, and domain services**.



Value Object

- A value object is an object that can be identified by the **composition** of its values. Example:

```
class color
{
    int _red;
    int _green;
    int _blue;
}
```

- The composition of the values of the three fields red, green, and blue defines a color.
- Changing the value of one of the fields will result in a **new** color.



Value Object (cont.)

- Redundant *ColorId* field, making it possible to have two rows with the same values:

Redundant

Colors

color-id	red	green	blue
1	255	255	0
2	0	128	128
3	0	0	255
4	0	0	255

Same color



Value Object (cont.)

```
class Person
{
    private int _id;
    private string _firstName;
    private string _lastName;
    private string _landlinePhone;
    private string _mobilePhone;
    private string _email;
    private int _heightMetric;
    private string _countryCode;

    public Person(...) {...}
}
static void Main(string[] args)
{
    var dave = new Person(
        id: 30217,
        firstName: "Dave",
        lastName: "Ancelovici",
        landlinePhone: "023745001",
        mobilePhone: "0873712503",
        email: "dave@learning-ddd.com",
        heightMetric: 180,
        countryCode: "BG");
}
```

```
class Person
{
    private PersonId _id;
    private Name _name;
    private PhoneNumber _landline;
    private PhoneNumber _mobile;
    private EmailAddress _email;
    private Height _height;
    private CountryCode _country;

    public Person(...) { ... }
}
static void Main(string[] args)
{
    var dave = new Person(
        id: new PersonId(30217),
        name: new Name("Dave", "Ancelovici"),
        landline: PhoneNumber.Parse("023745001"),
        mobile: PhoneNumber.Parse("0873712503"),
        email: Email.Parse("dave@learning-ddd.com"),
        height: Height.FromMetric(180),
        country: CountryCode.Parse("BG"));
}
```



Value Object (cont.)

- Problems of validating fields in class:
 1. The validation logic tends to be duplicated.
 2. It's hard to enforce calling the validation logic before the values are used.
- Alternative design by leveraging value objects:
 1. The value object makes the intent **clear**, even with shorter variable names.
 2. There is no need to validate the values before the assignment, as the validation logic resides in the value objects themselves.
 3. The **cohesive logic** is implemented in one place and is easy to test.
 4. Most importantly, value objects **express the business domain's concepts**: they make the code speak the ubiquitous language.



Value Object (cont.)

- The *PhoneNumber* value object can encapsulate the logic of:
 - Parsing a string value,
 - Validating it,
 - And extracting different attributes of the phone number; For example:
 - The country it belongs to.
 - The phone number's type—landline or mobile.

```
var phone = PhoneNumber.Parse("+359877123503");  
var country = phone.Country; // "BG"  
var phoneType = phone.PhoneType; // "MOBILE"  
var isValid = PhoneNumber.IsValid("+972120266680"); // false
```



Value Object Implementation

- Since a change to any of the fields of a value object results in a different value, value objects are implemented as **immutable objects**.
- A change to one of the value object's fields conceptually creates a different value—a **different instance** of a value object.

```
public class Color
{
    public readonly byte Red;
    public readonly byte Green;
    public readonly byte Blue;
    public Color(byte r, byte g, byte b)
    {
        this.Red = r;
        this.Green = g;
        this.Blue = b;
    }
    ...
}
```

```
public Color MixWith(Color other)
{
    return new Color(
        r: (byte) Math.Min(this.Red + other.Red, 255),
        g: (byte) Math.Min(this.Green + other.Green, 255),
        b: (byte) Math.Min(this.Blue + other.Blue, 255)
    );
    ...
}
```



When to Use Value Objects

- The simple answer is, **whenever you can**.
- Not only do value objects make the code **more expressive** and encapsulate business logic that tends to spread apart, but the pattern makes the code **safer**.
- Since value objects are **immutable**, the value objects' behavior is free of side effects and is **thread safe**.
- From a business domain perspective, a useful rule of thumb is to use value objects for the ***domain's elements that describe properties of other objects***.



Entities

- An **entity** is the opposite of a value object. It requires an explicit **identification field** to distinguish between the different instances of the entity.
- Example: An identification field is needed to properly identify people.
 - The Id can be a GUID, a number, a string, or a domain-specific value such as a Social Security number.

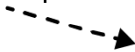
```
class Person
{
    public readonly PersonId Id;
    public Name Name { get; set; }
    public Person(PersonId id, Name name)
    {
        this.Id = id;
        this.Name = name;
    }
}
```



Entities (cont.)

- Contrary to value objects, entities are **not immutable** and are expected to change.
- Value objects **describe** an entity's properties.

Identification required



Id	First Name	Last Name
1	Tom	Cook
2	Harold	Elliot
3	Dianna	Daniels
4	Dianna	Daniels

Instance differentiation via ID



Aggregates

- We don't implement entities independently, but only in the context of the **aggregate pattern**.
- An aggregate is an *entity*: it requires an explicit **identification field** and its state is expected to **change** during an instance's lifecycle.
- The aggregate's logic has to **validate** all incoming modifications and ensure that the changes do not contradict its **business rules**.
- From an implementation perspective, the consistency is enforced by allowing **only** the aggregate's business logic to modify its state.



Aggregates (cont.)

- All processes or objects external to the aggregate are only allowed to read the aggregate's state.
- Its state can **only** be mutated by executing corresponding methods of the aggregate's **public interface**.
- The state-modifying methods exposed as an aggregate's public interface are often referred to as ***commands***.



Implementing a Command

- First, it can be implemented as a plain public method of the aggregate object:

```
public class Ticket
{
    ...
    public void AddMessage(UserId from,
        string body)
    {
        var message = new Message(from, body);
        _messages.Append(message);
    }
    ...
}
```

- Alternatively, a command can be represented as a parameter object that encapsulates all the input required for executing the command:

```
public class Ticket
{
    ...
    public void Execute(AddMessage cmd)
    {
        var message = new Message(cmd.from, cmd.body);
        _messages.Append(message);
    }
    ...
}
```



Implementing a Command (cont.)

- An aggregate's public interface is responsible for validating the input and enforcing all of the relevant business rules and invariants.
- This strict boundary also ensures that all business logic related to the aggregate is implemented in one place: **the aggregate itself**.
- This makes the **application layer** that orchestrates operations on aggregates rather simple: all it has to do is:
 1. **Load** the aggregate's current state,
 2. **Execute** the required action,
 3. **Persist** the modified state,
 4. And **return** the operation's result to the caller.



Consistency Enforcement

- It's vital to protect the consistency of an aggregate's **state**.
- If multiple processes are concurrently updating the same aggregate, we have to prevent the latter transaction from blindly **overwriting** the changes committed by the first one.
- Hence, the database used for storing aggregates has to support **concurrency management**.
- In its simplest form, an aggregate should hold a **version field** that will be incremented after each update.



Consistency Enforcement (cont.)

```
public ExecutionResult Escalate(TicketId id, EscalationReason reason)
{
    try
    {
        var ticket = _ticketRepository.Load(id);
        var cmd = new Escalate(reason);
        ticket.Execute(cmd);
        _ticketRepository.Save(ticket);
        return ExecutionResult.Success();
    }
    catch (ConcurrencyException ex)
    {
        return ExecutionResult.Error(ex);
    }
}
```

- *ConcurrencyException: An error that happens when multiple processes or threads try to modify the same data at the same time, and the system detects a conflict.*



Transaction Boundary

- Since an aggregate's state can **only** be modified by its own business logic, the aggregate also acts as a transactional boundary.
- All changes to the aggregate's state should be **committed transactionally** as one **atomic** operation.
- The need to commit changes in multiple aggregates signals a wrong transaction boundary, and hence, wrong aggregate boundaries.



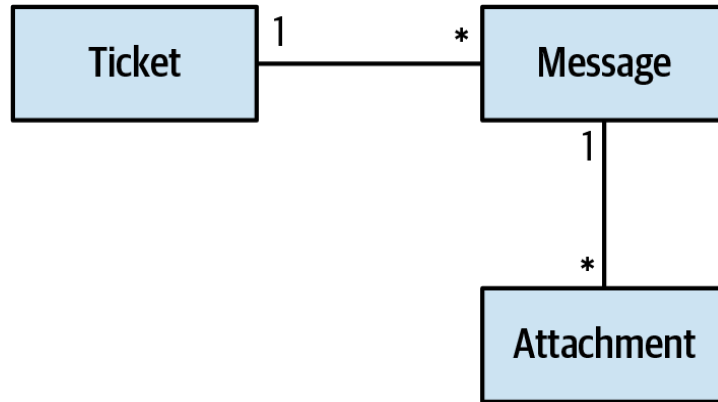
Hierarchy of Entities

- *What if we need to modify multiple objects in the same transaction?*
- As discussed earlier, we don't use entities as an independent pattern, only as **part of** an aggregate.
- There are business scenarios in which multiple objects should **share** a transactional boundary; for example, when both can be modified simultaneously or the business rules of one object depend on the state of another object.



Hierarchy of Entities (cont.)

- To support changes to multiple objects that have to be applied in one atomic transaction, the aggregate pattern resembles a hierarchy of entities, all sharing transactional consistency (as shown in the figure).



Aggregate as a hierarchy of entities



Hierarchy of Entities (cont.)

- The rule of thumb is to keep the aggregates as **small** as possible and include only objects that are required to be in a strongly consistent state by the aggregate's business logic.



Domain Events

- A domain event is a **message** describing a **significant event** that has occurred in the business domain. For example:
 - Ticket assigned
 - Ticket escalated
 - Message received
- Since domain events describe something that has already happened, their names should be formulated in the **past tense**.
- The goal of a domain event is to describe **what has happened** in the business domain and provide all the **necessary data** related to the event.



Domain Events (cont.)

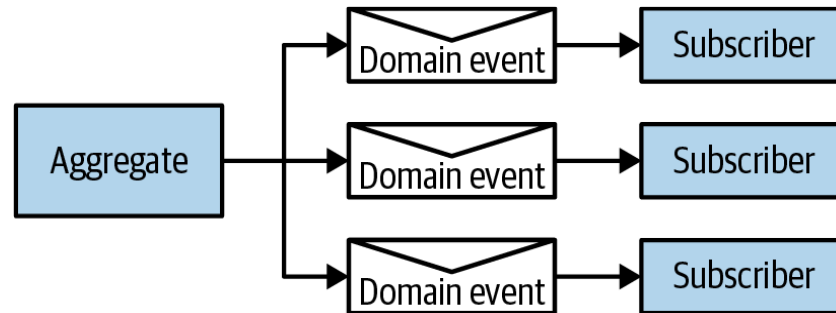
- Example: Following event indicates that a support ticket was escalated.

```
{  
  "ticket-id": "c9d286ff-3bca-4f57-94d4-4d4e490867d1",  
  "event-id": 146,  
  "event-type": "ticket-escalated",  
  "escalation-reason": "missed-sla",  
  "escalation-time": 1628970815  
}
```



Domain Events (cont.)

- Domain events are part of an aggregate's **public interface**.
- An aggregate **publishes** its domain events.
- Other processes, aggregates, or even external systems can **subscribe** to and **execute** their own logic in response to the domain events, as shown in the figure.



Domain events publishing flow



Domain Services

- Sooner or later, you may encounter business logic that either **doesn't** belong to any aggregate or value object, or that seems to be relevant to **multiple** aggregates.
- In such cases, domain-driven design proposes to implement the logic as a ***domain service***.
- A ***domain service*** is a **stateless** object that implements the business logic.



Domain Services (cont.)

- Example: Calculate the deadline for an agent to respond to a ticket.

```
public class ResponseTimeFrameCalculationService
{
    ...
    public ResponseTimeframe CalculateAgentResponseDeadline(UserId agentId,
        Priority priority, bool escalated, DateTime startTime)
    {
        var policy = _departmentRepository.GetDepartmentPolicy(agentId);
        var maxProcTime = policy.GetMaxResponseTimeFor(priority);
        if (escalated)
        {
            maxProcTime = maxProcTime * policy.EscalationFactor;
        }
        var shifts = _departmentRepository.GetUpcomingShifts(agentId,
            startTime, startTime.Add(policy.MaxAgentResponseTime));
        return CalculateTargetTime(maxProcTime, shifts);
    }
    ...
}
```



Domain Services (cont.)

- Domain services make it easy to **coordinate** the work of multiple aggregates.
- Domain services lend themselves to implementing calculation logic that requires **reading** the data of multiple aggregates.
- It is also important to point out that domain services have nothing to do with microservices, It is just a **stateless object** used to host business logic.



Repository

- A **Repository** is an abstraction that provides access to **domain objects (usually Aggregates)** as if they were stored in memory, while hiding the details of persistence (database, API, etc.).
- The repository's job is to:
 - Retrieve aggregates from storage
 - Save aggregates back to storage
 - Hide persistence details from the domain
- Clean Architecture places the repository **interface** in the domain or application layer, while the **implementation** lives in the infrastructure layer.



Conclusion

- The domain model's building blocks tackle the complexity of the business logic by encapsulating it in the boundaries of *value objects* and *aggregates*.
- The inability to modify the objects' state externally ensures that all the relevant business logic is implemented in the boundaries of aggregates and value objects and won't be duplicated in the **application layer**.



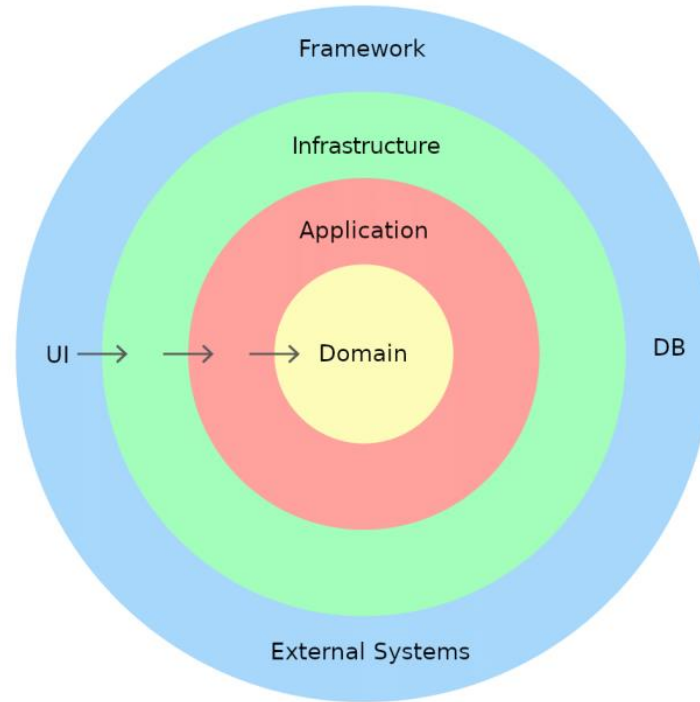
Clean Architecture and DDD

- **DDD** helps you model business complexity.
- **Clean Architecture** helps you organize software structure and dependencies.
- **Domain Model** is the bridge between them.

- DDD often fits naturally inside Clean Architecture.
 - **Clean Architecture = skeleton**
 - **DDD = brain/business knowledge**
- DDD tactical patterns usually live in the **Domain layer** and sometimes Application layer.



Clean Architecture and DDD (cont.)



Clean Architecture and DDD (cont.)

Clean Architecture Layer	DDD Concepts/Patterns
Frameworks / Infrastructure (Handles external technologies and implementation details)	- Repository Implementations - Anti-Corruption Layer (ACL) - External integrations
Interface Adapters (Converts external input/output formats to internal models)	- DTO mapping - Translators between bounded contexts - ACL adapters
Application / Use Case Layer (Orchestrates workflows and application-specific operations)	- Application Services - Command Handlers - Query Handlers - Domain Event Handlers
Domain Layer (Contains core business logic and business rules)	- Value Objects - Entities - Aggregates, Aggregate Roots - Domain Services, Domain Events



Modeling the Dimension of Time



Event-Sourced Domain Model

- The event-sourced domain model pattern is based on the same premise as the domain model pattern.
- Again, the business logic is **complex** and belongs to a **core subdomain**.
- Moreover, it uses the same tactical patterns as the domain model:
 - Value objects, aggregates, and domain events.
- The difference between these implementation patterns lies in the way the aggregates' **state** is persisted.
- Instead of persisting an aggregate's state, the model generates **domain events** describing each change and uses them as the source of truth for the aggregate's data.



Event Sourcing

- Examine following table and analyze what you can learn from this data about the system it belongs to.

lead-id	first-name	last-name	status	phone-number	followup-on	created-on	updated-on
1	Sean	Callahan	CONVERTED	555-1246		2019-01-31T10:02:40.32Z	2019-01-31T10:02:40.32Z
2	Sarah	Estrada	CLOSED	555-4395		2019-03-29T22:01:41.44Z	2019-03-29T22:01:41.44Z
3	Stephanie	Brown	CLOSED	555-1176		2019-04-15T23:08:45.59Z	2019-04-15T23:08:45.59Z
4	Sami	Calhoun	CLOSED	555-1850		2019-04-25T05:42:17.07Z	2019-04-25T05:42:17.07Z
5	William	Smith	CONVERTED	555-3013		2019-05-14T04:43:57.51Z	2019-05-14T04:43:57.51Z



Event Sourcing (cont.)

- It's evident that the table is used to manage potential customers, or leads, in a telemarketing system.
- By examining the various statuses, we can also assume the processing cycle each potential customer goes through.
- *But what information is missing from that table?*
- It misses the story of how each lead got to their current state.
- One of the ways to fill in the missing information is to use **event sourcing**.



Event Sourcing (cont.)

- The event sourcing pattern introduces the **dimension of time** into the data model.
- Instead of the schema reflecting the aggregates' current state, an event sourcing– based system persists **events**.
 - Documenting every change in an aggregate's lifecycle.



Example

```
{
  "lead-id": 12,
  "event-id": 0,
  "event-type": "lead-initialized",
  "first-name": "Casey",
  "last-name": "David",
  "phone-number": "555-2951",
  "timestamp": "2020-05-20T09:52:55.95Z"
},
{
  "lead-id": 12,
  "event-id": 1,
  "event-type": "contacted",
  "timestamp": "2020-05-20T12:32:08.24Z"
},
{
  "lead-id": 12,
  "event-id": 2,
  "event-type": "followup-set",
  "followup-on": "2020-05-27T12:00:00.00Z",
  "timestamp": "2020-05-20T12:32:08.24Z"
}
```

```
{
  "lead-id": 12,
  "event-id": 3,
  "event-type": "contact-details-updated",
  "first-name": "Casey",
  "last-name": "Davis",
  "phone-number": "555-8101",
  "timestamp": "2020-05-20T12:32:08.24Z"
},
{
  "lead-id": 12,
  "event-id": 4,
  "event-type": "contacted",
  "timestamp": "2020-05-27T12:02:12.51Z"
},
{
  "lead-id": 12,
  "event-id": 5,
  "event-type": "order-submitted",
  "payment-deadline": "2020-05-30T12:02:12.51Z",
  "timestamp": "2020-05-27T12:02:12.51Z"
},
{
  "lead-id": 12,
  "event-id": 6,
  "event-type": "payment-confirmed",
  "status": "converted",
  "timestamp": "2020-05-27T12:38:44.12Z"
}
```



Implementation

```
public class LeadSearchModelProjection
{
    public long LeadId { get; private set; }
    public HashSet<string> FirstNames { get; private set; }
    public HashSet<string> LastNames { get; private set; }
    public HashSet<PhoneNumber> PhoneNumbers
    { get; private set; }
    public int Version { get; private set; }
    public void Apply(LeadInitialized @event)
    {
        LeadId = @event.LeadId;
        FirstNames = new HashSet<string>();
        LastNames = new HashSet<string>();
        PhoneNumbers = new HashSet<PhoneNumber>();
        FirstNames.Add(@event.FirstName);
        LastNames.Add(@event.LastName);
        PhoneNumbers.Add(@event.PhoneNumber);
        Version = 0;
    }
}
```

```
public void Apply(ContactDetailsChanged @event)
{
    FirstNames.Add(@event.FirstName);
    LastNames.Add(@event.LastName);
    PhoneNumbers.Add(@event.PhoneNumber);
    Version += 1;
}

public void Apply(Contacted @event)
{ Version += 1; }

public void Apply(FollowupSet @event)
{ Version += 1; }

public void Apply(OrderSubmitted @event)
{ Version += 1; }

public void Apply(PaymentConfirmed @event)
{ Version += 1; }
}
```



Implementation (cont.)

- The customer's state can easily be projected out from these domain events.
- Iterating an aggregate's events and feeding them sequentially into the appropriate overrides of the *Apply* method will produce precisely the state representation model.
- Pay attention to the Version field that is incremented after applying each event.
- The event store should not allow modifying or deleting the events since it's append-only storage.



Deducing current state

- In essence, the event sourcing pattern **is nothing new**.
- The **financial industry** uses events to represent changes in a **ledger**.
- A ledger is an append-only log that documents transactions.
- A current state (e.g., account balance) can always be **deduced** by “projecting” the ledger’s records.



Event-Sourced Domain Model

- The original domain model maintains a **state** representation of its aggregates and emits select domain events.
- The event-sourced domain model uses domain events exclusively for modeling the **aggregates' lifecycles**.
- All changes to an aggregate's state have to be expressed as domain events.



Event-Sourced Domain Model (cont.)

- Each operation on an event-sourced aggregate follows this script:
 1. Load the aggregate's domain events.
 2. Reconstitute a state representation—project the events into a state representation that can be used to make business decisions.
 3. Execute the aggregate's command to execute the business logic, and consequently, produce new domain events.
 4. Commit the new domain events to the event store.



Event Sourcing Advantages

- *Time traveling*
 - They can be used to restore all **past states** of the aggregate.
- *Deep insight*
 - Event sourcing provides deep insight into the system's state and behavior.
- *Audit log*
 - The persisted domain events represent audit log of everything that has **happened** to the aggregates' states.
 - This model is especially convenient for systems managing **money** or **monetary transactions**.
- *Advanced optimistic concurrency management*



Event Sourcing Disadvantages

- *Learning curve*
 - Successful implementation of the pattern demands training of the team and time to get used to the new way of thinking.
- *Evolving the model*
 - Evolving an event-sourced model can be challenging.
- *Architectural complexity*



References

- Learning Domain-Driven Design, *Vlad Khnonov*.
 - Chapters 5, 6, 7

